

# PE-EC802C — VLSI Design Automation

Semester VIII • 3L:0T:0P • 3 Credits

## Module 1: Introduction to VLSI Design Methodologies & Algorithms

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

---

### Contents

---

|          |                                                           |          |
|----------|-----------------------------------------------------------|----------|
| <b>1</b> | <b>VLSI Design Flow &amp; Abstraction Levels</b>          | <b>2</b> |
| 1.1      | Gajski-Kuhn Y-Chart                                       | 2        |
| 1.2      | Physical Design Flow Phases                               | 3        |
| 1.3      | VLSI Design Automation Tools Overview                     | 3        |
| <b>2</b> | <b>Algorithmic Graph Theory &amp; Complexity</b>          | <b>3</b> |
| 2.1      | Graph Representation Models                               | 3        |
| 2.2      | Bipartite Graphs in VLSI                                  | 4        |
| 2.3      | Tractability and Complexity Classes                       | 4        |
| 2.4      | Data Structures & Algorithms in VLSI CAD                  | 4        |
| <b>3</b> | <b>Combinatorial Optimization &amp; Heuristic Methods</b> | <b>5</b> |
| 3.1      | 1. Simulated Annealing (SA)                               | 5        |
| 3.1.1    | Algorithmic Flow                                          | 5        |
| 3.2      | 2. Genetic Algorithms (GA)                                | 6        |
| 3.3      | 3. Integer Linear Programming (ILP)                       | 7        |

## 1. VLSI Design Flow & Abstraction Levels

### VLSI Design Automation

**VLSI Design Automation** refers to the use of computer-aided design (CAD) software tools to automate the translation of a high-level description of an electronic system into a physical layout fabricated on a silicon wafer. It covers synthesis, optimization, physical layout, verification, and testing.

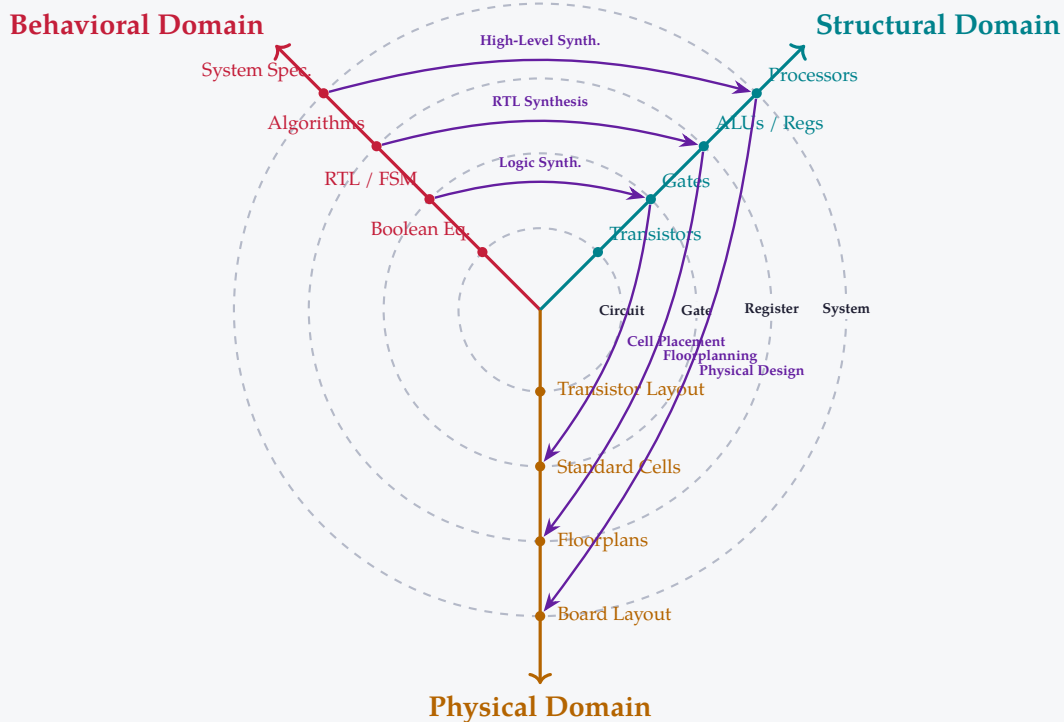
### 1.1 Gajski-Kuhn Y-Chart

The Gajski-Kuhn Y-Chart is a framework that describes electronic system design using three distinct domains represented as three axes of a Y:

- **Behavioral Domain:** Describes *what* the system does in terms of function, transfer functions, executable algorithms, and state machines.
- **Structural Domain:** Describes *how* the system is connected in terms of hardware entities like registers, ALUs, logic gates, and transistors.
- **Physical Domain:** Describes the physical implementation details on silicon, including floorplans, cell placement, macro routing, and transistor layouts.

The concentric rings crossing the three axes represent the **levels of abstraction**: System, Register Transfer Level (RTL), Gate/Logic, and Circuit/Transistor levels.

### Gajski-Kuhn Y-Chart



## 1.2 Physical Design Flow Phases

Physical design is the process of converting the synthesized netlist into a geometric layout ready for fabrication. The phases include:

1. **Partitioning:** Dividing a large system into smaller, manageable blocks to minimize communication overhead and simplify design.
2. **Floorplanning & Sizing:** Positioning major blocks on the silicon die, assigning aspect ratios, and planning power/ground distribution.
3. **Placement:** Assigning exact geometric coordinates to standard cells inside the floorplanned blocks to minimize total wirelength.
4. **Routing:** Connecting standard cells and pads using metal wire tracks. This is divided into *Global Routing* (allocating rough paths) and *Detailed Routing* (assigning wires to specific metal layers and tracks).
5. **Compaction:** Shrinking the layout to its absolute minimum size while adhering to the technological design rules (spacing, width rules).

## 1.3 VLSI Design Automation Tools Overview

Modern VLSI design flows rely heavily on Electronic Design Automation (EDA) tools from major vendors (such as Synopsys, Cadence, and Siemens EDA) as well as open-source projects. These tools automate different stages of the design cycle:

| Design Phase          | Commercial / Industry Standard             | Academic / Open-Source    |
|-----------------------|--------------------------------------------|---------------------------|
| Logic Synthesis       | Synopsys Design Compiler, Cadence Genus    | Yosys, ABC                |
| Placement & Routing   | Synopsys IC Compiler II, Cadence Innovus   | OpenROAD, Graywolf        |
| Physical Verification | Siemens EDA Calibre, Synopsys IC Validator | Magic, Netgen             |
| Simulation            | Synopsys VCS, Cadence Xcelium, ModelSim    | Icarus Verilog, Verilator |

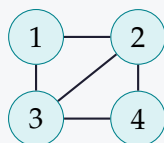
## 2. Algorithmic Graph Theory & Complexity

Physical design layouts are represented abstractly using graph data structures. For example, standard cells are represented as vertices ( $V$ ), and the interconnect wires are represented as edges ( $E$ ).

### 2.1 Graph Representation Models

- **Adjacency Matrix ( $A$ ):** A 2D array of size  $|V| \times |V|$  where  $A[i][j] = 1$  if there is an edge between  $v_i$  and  $v_j$ , and 0 otherwise. Suitable for dense graphs ( $|E| \approx |V|^2$ ).
- **Adjacency List:** An array of lists of size  $|V|$ , where index  $i$  stores the neighbors of vertex  $v_i$ . Memory efficient for sparse graphs ( $|E| \ll |V|^2$ ).

## Graph Representation Example



Adjacency Matrix:

|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 |
| 2 | 1 | 0 | 1 | 1 |
| 3 | 1 | 1 | 0 | 1 |
| 4 | 0 | 1 | 1 | 0 |

Adjacency List:

| Node | Neighbors |
|------|-----------|
| 1    | 2 → 3     |
| 2    | 1 → 3 → 4 |
| 3    | 1 → 2 → 4 |
| 4    | 2 → 3     |

## 2.2 Bipartite Graphs in VLSI

A graph  $G = (V, E)$  is **bipartite** if its vertices can be partitioned into two disjoint sets  $U$  and  $W$  such that every edge  $e \in E$  connects a vertex in  $U$  to a vertex in  $W$  (no edges exist between vertices of the same set). Bipartite graphs are used to represent:

- Pin-to-net connectivity lists in floorplanning.
- Left-Edge algorithm horizontal/vertical routing constraints.
- Hardware binding/allocation problems.

## 2.3 Tractability and Complexity Classes

VLSI design problems involve thousands of components, making algorithm efficiency critical. We classify problems into complexity classes:

- **Class P:** Problems solvable in polynomial time  $O(n^k)$  by a deterministic Turing machine. Example: Shortest path routing.
- **Class NP:** Decision problems whose solutions are verifiable in polynomial time by a deterministic machine (or solvable in polynomial time by a non-deterministic Turing machine).
- **NP-Complete:** The hardest problems in Class NP. If a polynomial-time algorithm exists for any NP-complete problem, then  $P = NP$ . Example: Partitioning, Floorplanning, and Routing.
- **NP-Hard:** Problems at least as hard as any NP-complete problem, but not necessarily in NP (often optimization problems).

Since most physical design steps (partitioning, placement, routing) are NP-hard optimization problems, VLSI CAD tools must use **heuristic algorithms** (like simulated annealing, genetic algorithms) to find near-optimal solutions in reasonable computing time.

## 2.4 Data Structures & Algorithms in VLSI CAD

VLSI design automation tools utilize classic computer science data structures and algorithms to model, analyze, and optimize physical circuits:

| Concept                          | VLSI Application / Example                                      | Core Role in Automation                                         |
|----------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Graphs &amp; Hypergraphs</b>  | Netlists, routing grids, horizontal/vertical constraint graphs. | Models pin connectivity and track ordering.                     |
| <b>Adjacency Matrix/List</b>     | Grid representation, component connection lists.                | Back-end database for topological analysis.                     |
| <b>Trees (Steiner / Slicing)</b> | Global routing trees, slicing floorplan blocks, BDDs.           | Optimizes wirelength, layout structures, and Boolean functions. |
| <b>Priority Queues (Heaps)</b>   | Time-wheel event simulation, list scheduling priority queues.   | Tracks next scheduled simulation events or operations.          |
| <b>Dijkstra / BFS / DFS</b>      | Maze routing (Lee's algorithm), reachability analysis.          | Finds shortest routing paths around layout obstacles.           |
| <b>Graph Coloring</b>            | Register binding / allocation during HLS.                       | Assigns registers to non-overlapping variable lifetimes.        |

### 3. Combinatorial Optimization & Heuristic Methods

#### 3.1 1. Simulated Annealing (SA)

Simulated Annealing is a stochastic global optimization technique inspired by the process of physical annealing in metallurgy, where a metal is heated to a high temperature and cooled slowly to reach a low-energy, highly ordered crystalline state.

##### Simulated Annealing Core Principle

To escape local minima, Simulated Annealing occasionally accepts worse moves (uphill steps) with a probability that decreases as the temperature decreases. This probability is governed by the Boltzmann distribution:

$$P(\text{accept}) = e^{-\frac{\Delta E}{T}} \quad (1)$$

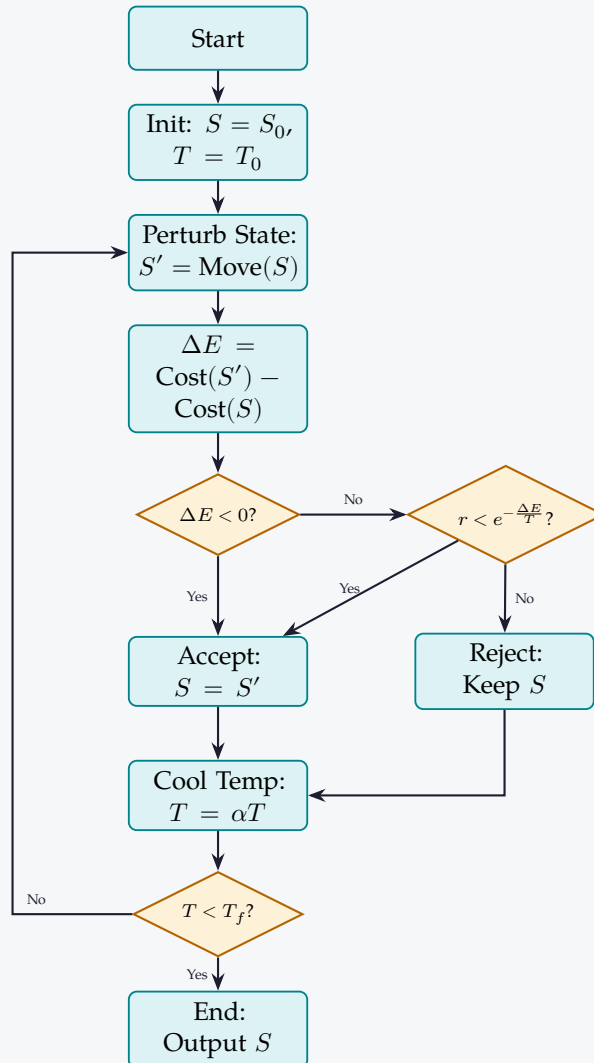
where  $\Delta E = \text{Cost}(S_{\text{new}}) - \text{Cost}(S_{\text{current}})$  is the change in cost, and  $T$  is the current temperature.

##### 3.1.1 Algorithmic Flow

- Initialization:** Start with an initial configuration  $S_0$  and a high initial temperature  $T_0$ .
- Perturbation:** Make a small random change to the configuration to generate a neighbor state  $S'$ .
- Cost Evaluation:** Compute  $\Delta E = \text{Cost}(S') - \text{Cost}(S)$ .
- Acceptance Criteria:**
  - If  $\Delta E < 0$  (improvement), accept the new state ( $S \leftarrow S'$ ).
  - If  $\Delta E \geq 0$  (worse state), accept  $S'$  only if a random number  $r \in [0, 1)$  satisfies  $r < e^{-\frac{\Delta E}{T}}$ . Otherwise, reject it and keep  $S$ .

5. **Cooling:** Reduce the temperature using a cooling function (typically  $T_{k+1} = \alpha T_k$ , where  $0.85 \leq \alpha \leq 0.98$ ).
6. **Termination:** Repeat until  $T$  reaches the stopping temperature  $T_f$  or the system freezes.

Simulated Annealing Flow Chart



### 3.2 2. Genetic Algorithms (GA)

Genetic Algorithms are population-based heuristics inspired by natural selection and evolutionary biology.

- **Representation:** Designs (e.g., placements) are encoded as strings of symbols called chromosomes.
- **Evaluation:** A fitness function determines the quality of each chromosome (e.g., inverse of total wirelength).
- **Crossover:** Pairs of chromosomes (parents) are combined with a probability  $P_c$  to create offspring.
- **Mutation:** Random bits in the offspring chromosomes are inverted with a small probability  $P_m$  to maintain genetic diversity and prevent premature convergence.
- **Selection:** Fitter individuals are selected to form the next generation.

### 3.3 3. Integer Linear Programming (ILP)

ILP is a mathematical optimization method where both the objective function and the constraints are linear equations, and the decision variables are restricted to integers.

- Used for exact placement, routing track allocation, and resource scheduling.
- Yields the global optimum but suffers from high computational complexity (NP-complete), making it practical only for small design subproblems.

## Quick Revision — Module 1

Introduction to VLSI Design Automation | PE-EC802C

### Key Formulas and Metrics at a Glance

|                                   |                                                                                      |
|-----------------------------------|--------------------------------------------------------------------------------------|
| <b>Boltzmann Probability</b>      | $P(\text{accept}) = e^{-\frac{\Delta E}{T}}$ (accepts uphill moves when $T$ is high) |
| <b>Geometric Cooling Schedule</b> | $T_{k+1} = \alpha T_k$ ( $0.85 \leq \alpha \leq 0.98$ )                              |
| <b>Degree Sum Theorem</b>         | $\sum_{v \in V} \deg(v) = 2 E $ (Handshaking Lemma)                                  |

### One-Line Definitions

- **Y-Chart:** A representation of hardware design along Behavioral, Structural, and Physical axes.
- **Heuristic:** An approximate algorithm designed to solve NP-hard optimization problems in polynomial time.
- **Bipartite Graph:** A graph whose vertices are split into two disjoint sets, with edges only running between them.
- **Tractability:** The property of a problem indicating whether it can be solved within polynomial time.
- **Crossover:** The genetic algorithm operator that merges features of two parent chromosomes to create offspring.
- **Compaction:** The physical design phase that squeezes layouts to minimum area without violating design rules.

### Frequently Asked / Viva Questions

#### Q1. Why is physical design automation necessary in modern chip design?

Modern chips contain billions of transistors. Manual design is impossible due to complexity, human error, and time-to-market constraints. Automation ensures optimization, design rule compliance, and correct-by-construction layout generation within a few days.

#### Q2. What are the typical cost components optimized during cell placement?

The placement phase optimizes: (1) Total wirelength (usually estimated using half-perimeter wirelength or HPWL), (2) Chip bounding box area, (3) Routing congestion, and (4) Critical path timing delays.

#### Q3. Explain the metallurgical analogy used in Simulated Annealing.

In physical annealing, metal is heated to melt bonds (high energy, random atom states) and cooled very slowly. Atoms settle into a stable crystal lattice

**Frequently Asked / Viva Questions (contd.)**

(minimum energy state). In Simulated Annealing, the configuration represents atom positions, cost represents system energy, and the control parameter  $T$  acts as the temperature. High  $T$  allows random moves, while slow cooling lets the algorithm settle into a global minimum cost state.

**Q4. What is the difference between NP-complete and NP-hard problems?**

NP-complete problems belong to class NP (their solutions can be verified in polynomial time) and are at least as hard as any other problem in NP. NP-hard problems are not necessarily in NP (verification might not run in polynomial time), but any NP-complete problem can be reduced to them in polynomial time.

**Q5. Why can't we use exact methods like Integer Linear Programming (ILP) for whole-chip routing?**

ILP is an NP-complete problem. The execution time of ILP solvers increases exponentially with the number of integer variables. Since modern chips contain millions of nets, solving a single global ILP formulation would take years. Hence, heuristics are used instead.

**Q6. How does the mutation parameter prevent premature convergence in Genetic Algorithms?**

Premature convergence occurs when the population becomes uniform, trapped in a local minimum. Mutation randomly alters individual chromosome bits, reintroducing lost genetic traits and letting the algorithm explore untested search space regions.

**Q7. What is a hypergraph, and how is it used in circuit partitioning?**

A hypergraph is a generalization of a graph where an edge (called a hyperedge) can connect any number of vertices. In VLSI, standard cells are vertices, and multi-terminal nets connecting three or more pins are represented as hyperedges.

**Q8. What role does the parameter  $\alpha$  play in Simulated Annealing?**

The parameter  $\alpha$  is the cooling rate. If  $\alpha$  is too small (e.g. 0.5), the temperature falls too rapidly, causing the system to freeze into a poor local minimum. If  $\alpha$  is too close to 1 (e.g. 0.999), the cooling is extremely slow, and the algorithm takes too long to run. Standard values are between 0.85 and 0.98.

**Q9. What is the difference between top-down and bottom-up design methodologies?**

Top-down starts with a high-level specification and refines it down to RTL, gate-level, and finally transistor layout. Bottom-up starts by designing individual custom circuits, standard cells, or sub-blocks, and then joins them to build the complete system.

**Q10. Why does routing require two separate steps: Global Routing and Detailed Routing?**

Routing is highly complex. Solving it in one step is computationally impossible. Global routing simplifies the problem by dividing the chip into a coarse grid

**Frequently Asked / Viva Questions (contd.)**

and routing nets through grid regions without assigning specific tracks. Detailed routing then focuses on each grid region separately to assign nets to precise metal tracks and layers.

**Comparison Snapshot****Structural vs. Behavioral Domain:**

| Feature        | Behavioral Domain                                           | Structural Domain                                         |
|----------------|-------------------------------------------------------------|-----------------------------------------------------------|
| Description    | Focuses on algorithmic functionality and logic transitions. | Focuses on hardware components and connectivity netlists. |
| Example        | Transfer functions, state equations, Boolean expressions.   | Logic gates, flip-flops, ALUs, multiplexers.              |
| Representation | VHDL/Verilog process blocks, C/C++ behavior models.         | Structural netlists, schematic diagrams.                  |

**Class P vs. Class NP Problems:**

| Feature      | Class P                                                 | Class NP                                                    |
|--------------|---------------------------------------------------------|-------------------------------------------------------------|
| Solvability  | Solvable in polynomial time by a deterministic machine. | Solvable in polynomial time by a non-deterministic machine. |
| Verification | Can be solved and verified in polynomial time.          | Solutions can be verified in polynomial time.               |
| VLSI Example | Shortest-path routing (Dijkstra's).                     | Cell placement, partition minimization, floorplan sizing.   |